

semmcci: Monte Carlo Confidence Intervals

Ivan Jacob Agaloos Pesigan

Installation

You can install the CRAN release of `semmcci` with:

```
install.packages("semmcci")
```

You can install the development version of `semmcci` from [GitHub](#) with:

```
if (!require("pak")) install.packages("pak")
pak::pkg_install("jeksterslab/semmcci")
```

Documentation

See [GitHub Pages](#) for package documentation.

Description

In the Monte Carlo method, a sampling distribution of parameter estimates is generated from the multivariate normal distribution using the parameter estimates and the sampling variance-covariance matrix. Confidence intervals for defined parameters are generated by obtaining percentiles corresponding to $100(1 - \alpha)\%$ from the generated sampling distribution, where α is the significance level.

Monte Carlo confidence intervals for free and defined parameters in models fitted in the structural equation modeling package `lavaan` can be generated using the `semmcci` package. The package has two main functions, namely, `MC()` and `MCStd()`. The output of `lavaan` is passed as the first argument to the `MC()` function to generate Monte Carlo confidence intervals. Monte Carlo confidence intervals for the standardized estimates can also be generated by passing the output of the `MC()` function to the `MCStd()` function. A description of the package and code examples are presented in Pesigan and Cheung (2024).

Example

A common application of the Monte Carlo method is to generate confidence intervals for the indirect effect. In the simple mediation model, variable **X** has an effect on variable **Y**, through a mediating variable **M**. This mediating or indirect effect is a product of path coefficients from the fitted model.

```
library(semmcci)
library(lavaan)
```

Data

```
n <- 1000
X <- rnorm(n = n)
M <- 0.50 * X + rnorm(n = n)
Y <- 0.25 * X + 0.50 * M + rnorm(n = n)
data <- data.frame(X, M, Y)
```

Model Specification

The indirect effect is defined by the product of the slopes of paths **X** to **M** labeled as **a** and **M** to **Y** labeled as **b**. In this example, we are interested in the confidence intervals of `indirect` defined as

the product of **a** and **b** using the `:=` operator in the **lavaan** model syntax.

```
model <- "  
  Y ~ cp * X + b * M  
  M ~ a * X  
  indirect := a * b  
  direct := cp  
  total := cp + (a * b)  
"
```

Model Fitting

We can now fit the model using the `sem()` function from **lavaan**.

```
fit <- sem(data = data, model = model)
```

Monte Carlo Confidence Intervals

The `fit` **lavaan** object can then be passed to the `MC()` function to generate Monte Carlo confidence intervals.

```
MC(fit, R = 20000L, alpha = 0.05)  
  
#> Monte Carlo Confidence Intervals  
#>      est      se      R  2.5% 97.5%  
#> cp      0.2191 0.0360 20000 0.1488 0.2898  
#> b      0.5201 0.0328 20000 0.4562 0.5856  
#> a      0.4592 0.0315 20000 0.3970 0.5212  
#> Y~~Y    0.9934 0.0441 20000 0.9080 1.0802  
#> M~~M    0.9400 0.0424 20000 0.8573 1.0235
```

```
#> X~~X      0.9361 0.0000 20000 0.9361 0.9361
#> indirect 0.2388 0.0221 20000 0.1970 0.2840
#> direct    0.2191 0.0360 20000 0.1488 0.2898
#> total     0.4579 0.0365 20000 0.3860 0.5282
```

Standardized Monte Carlo Confidence Intervals

Standardized Monte Carlo Confidence intervals can be generated by passing the result of the `MC()` function to `MCStd()`.

Note: We recommend setting `fixed.x = FALSE` when generating standardized estimates and confidence intervals to model the variances and covariances of the predictors if they are assumed to be random.

```
fit <- sem(data = data, model = model, fixed.x = FALSE)
unstd <- MC(fit, R = 20000L, alpha = 0.05)
vcov(unstd)
```

#>	cp	b	a	Y~~Y	M~~M
#> cp	1.257242e-03	-4.657792e-04	1.040589e-06	-4.001069e-06	-1.101279e-05
#> b	-4.657792e-04	1.040832e-03	-1.083553e-05	-5.551784e-06	1.033880e-05
#> a	1.040589e-06	-1.083553e-05	9.946767e-04	-8.682316e-06	1.673158e-06
#> Y~~Y	-4.001069e-06	-5.551784e-06	-8.682316e-06	1.990293e-03	8.511002e-06
#> M~~M	-1.101279e-05	1.033880e-05	1.673158e-06	8.511002e-06	1.785006e-03
#> X~~X	1.741559e-05	-7.144433e-06	-1.802001e-05	-7.243596e-06	-7.548483e-06
#> indirect	-2.129696e-04	4.720779e-04	5.123133e-04	-7.459458e-06	5.415222e-06
#> direct	1.257242e-03	-4.657792e-04	1.040589e-06	-4.001069e-06	-1.101279e-05
#> total	1.044272e-03	6.298760e-06	5.133539e-04	-1.146053e-05	-5.597566e-06
#>	X~~X	indirect	direct	total	
#> cp	1.741559e-05	-2.129696e-04	1.257242e-03	1.044272e-03	

```
#> b      -7.144433e-06  4.720779e-04 -4.657792e-04  6.298760e-06
#> a      -1.802001e-05  5.123133e-04  1.040589e-06  5.133539e-04
#> Y~~Y    -7.243596e-06 -7.459458e-06 -4.001069e-06 -1.146053e-05
#> M~~M    -7.548483e-06  5.415222e-06 -1.101279e-05 -5.597566e-06
#> X~~X     1.738251e-03 -1.317973e-05  1.741559e-05  4.235863e-06
#> indirect -1.317973e-05  4.841494e-04 -2.129696e-04  2.711798e-04
#> direct   1.741559e-05 -2.129696e-04  1.257242e-03  1.044272e-03
#> total    4.235863e-06  2.711798e-04  1.044272e-03  1.315452e-03
```

MCStd(unstd)

```
#> Standardized Monte Carlo Confidence Intervals
#>      est      se      R  0.05%   0.5%   2.5%  97.5%  99.5%  99.95%
#> cp      0.1764 0.0283 20000 0.0835 0.1023 0.1204 0.2318 0.2472 0.2663
#> b      0.4616 0.0261 20000 0.3747 0.3929 0.4097 0.5121 0.5276 0.5474
#> a      0.4166 0.0259 20000 0.3293 0.3490 0.3649 0.4664 0.4816 0.5004
#> Y~~Y    0.6880 0.0243 20000 0.6081 0.6234 0.6393 0.7343 0.7486 0.7647
#> M~~M    0.8265 0.0216 20000 0.7496 0.7681 0.7825 0.8669 0.8782 0.8916
#> X~~X    1.0000 0.0000 20000 1.0000 1.0000 1.0000 1.0000 1.0000 1.0000
#> indirect 0.1923 0.0164 20000 0.1416 0.1515 0.1603 0.2248 0.2357 0.2505
#> direct   0.1764 0.0283 20000 0.0835 0.1023 0.1204 0.2318 0.2472 0.2663
#> total    0.3687 0.0272 20000 0.2745 0.2959 0.3142 0.4209 0.4378 0.4548
```

References

Pesigan, I. J. A., & Cheung, S. F. (2024). Monte Carlo confidence intervals for the indirect effect with missing data. *Behavior Research Methods*, 56(3), 1678–1696. <https://doi.org/10.3758/s13428-023-02114-4>

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://www.R-project.org/>